Foodie

Online Food Service

Developed By: **Karam Ayoub**

Issued On: **Jan 2026**

Foodie – Online Food Service

Foodie is a web-based food ordering system that allows users to browse restaurants, reserve tables, place orders, and complete payments online.

It is built as a full-stack application with authentication, order management, and payment handling.

Features

- ✓ User authentication and authorization
- ✓ Browse restaurants and menus
- ✓ Add items to cart and place orders
- ✓ Reserve tables online
- ✓ Online and cash-on-delivery payments
- ✓ Admin dashboard for order management

Tech Stack

Backend: Laravel 10 (PHP)

Frontend: Blade / Bootstrap

Database: MySQL

Authentication: Laravel Auth

Testing: Unit & Feature

Installation

1. Clone the repository
   Run *"git clone https://gitlab.com/karam_ayoub/foodie.git"* on command line.

2. Install dependencies
   Run *"composer install"* on command line.

3. Copy environment file (.env.example)

4. Generate app key
   Run *"php artisan key:generate"* on command line.

5. Import data from database file (foodie.sql)

dashboard login:	user login:
Username: admin *Password: 12345678*	*Username: test_user* *Password: 123123*

6. Start the server
   Run *"php artisan serve"* on command line.

Testing

Feature Testing

Payment Test:

This project includes automated tests to verify the correctness and reliability of the payment workflow. The following test cases ensure that different payment methods behave as expected and that validation errors are handled properly:

- `test_perform_payment_using_credit_card_way_succeeded`
  Verifies that a payment performed using the **credit card** method is processed successfully and completes the checkout flow without errors.
- `test_perform_payment_using_cash_on_delivery_way_succeeded`
  Ensures that the **cash on delivery** payment option is accepted and that the order is successfully created using this method.
- `test_perform_payment_validation_error_redirects_back_to_cart`
  Confirms that when a **payment validation error** occurs, the user is correctly redirected back to the cart page, allowing them to review and fix the issue.

These tests help guarantee a stable and user-friendly payment experience by covering both successful payment scenarios and error-handling behavior.

Testing

Unit Testing

UUID Generation Test:

This unit test verifies that the UUID generated by the system is returned correctly and conforms to the expected standard format. It ensures that the generated UUID is a valid string with a length of **36 characters** and follows the **RFC 4122 UUID structure**, including hyphen placement and character validity.

By validating both the length and format of the UUID, this test guarantees the reliability and consistency of UUID generation across the application.

?. How to perform testing

In command line (terminal) execute this command:
"php artisan test" – to run all tests (Feature & Unit tests together).

OR

"php artisan test --filter=ExampleTest" – to run a specific test (ex: --filter=UuidTest).

Tests: [PaymentTest & UuidTest]

Thanks for your time